The great power of having this heuristic function is that we can use a different metric with respect to the distance, and convey more information about the domain; heuristics could even combine several data, as long as the value returned by a heuristic is appropriately scaled to make sense when compared to the edge's distance: for instance, it would make little sense (and cause terrible performance) to use meters for edges' weights and seconds (or even millimeters) for the heuristic's estimates; however, we can always scale millimeters to meters or, if the heuristic conveys information about the average time needed to reach the goal from each vertex, then we could multiply it for the roads' average or minimum speed, to obtain a quantity that could then be added to any edge's weight.

But we can also take this to the next level by decoupling even more the purposes of the distance and of the heuristic: imagine that we are computing the best route on the go, instead of a-priori, like a car navigator does.

First, we switch our metric from distance to travelling time: and the time needed to traverse a road changes depending on traffic, weather, areas closed to transit during some hours etc…

However, the average time needed when following a certain route is known in advance, and we can use that as a compass to balance our live decisions.

Suppose, for instance, that now you are also planning deliveries on a larger scale, and you need to ship some goods from Naples to Florence. When you get to Rome, the faster route would be through the motorway passing East of